

Sistema de Matchmaking para Jogos Online

José Pedro Dresch

Phelipe Gabriel Lima da Silva

Análise Teórica e de Custos das Operações

1. Implementação dos Algoritmos de Ordenação

Foram implementados dois algoritmos distintos para a ordenação dos jogadores por *score* e *timestamp*:

Insertion Sort: Implementado de forma iterativa, o algoritmo percorre o array a partir do segundo elemento, deslocando-o para a esquerda até encontrar sua posição correta em relação aos elementos já processados. É eficiente para conjuntos de dados pequenos ou quase ordenados.

Merge Sort: Utiliza a estratégia de "divisão e conquista". A implementação é baseada em **recursão**: a função `mergeSort` divide o array em duas metades sucessivamente até atingir o caso base (tamanho 1). Após o retorno da recursão, a função `merge` intercala os sub-arrays ordenados em um novo array auxiliar, garantindo a ordem global. A recursão permite que o problema seja quebrado em sub-problemas de complexidade logarítmica.

2. Estratégia para Formação de Grupos

A eficiência da função `formGroup` depende diretamente da ordenação prévia dos jogadores. Em um array ordenado por *score*, os jogadores com níveis de habilidade mais próximos estão necessariamente em posições adjacentes.

Para encontrar um grupo válido de tamanho k com uma diferença máxima Δ , o sistema realiza uma busca linear simples. Ele verifica apenas janelas de elementos consecutivos (da posição i até $i + k - 1$). Se a condição $score_{i+k-1} - score_i \leq \Delta$ for satisfeita, o grupo é considerado válido. Sem a ordenação, seria necessário testar todas as combinações possíveis de jogadores, o que faria a operação ser um caos computacional.

3. Análise de Custo Computacional

Abaixo, descrevemos a complexidade de tempo das principais operações no pior caso, onde n representa o número de jogadores na fila:

Operação	Complexidade	Justificativa
Inserção (<code>insert</code>)	$O(1)$	Inserção direta no final do array estático.
Remoção (<code>removePlayer</code>)	$O(n)$	Busca linear pelo ID e deslocar Players.
<i>Insertion Sort</i>	$O(n^2)$	Dois loops para comparar e deslocar.
<i>Merge Sort</i>	$O(n \log n)$	Divisão logarítmica e junção de partes.
Formação de Grupo	$O(n)$	Busca por janela e deslocar para remover.
Recuperação de Dados	$O(n)$	Cópia dos elementos para um novo array.

Tabela 1: Resumo da complexidade das operações.

Priorizamos o uso de memória dinâmica apenas quando necessário (como no retorno de arrays de grupos e na ordenação Merge), mantendo a estrutura principal em um array estático para respeitar as restrições do projeto.

Comparação Experimental de Desempenho

Para avaliar o desempenho prático dos algoritmos implementados, foram realizados testes com diferentes tamanho dos dados: 1.000, 10.000, 30.000, 50.000, 75.000 e 100.000 jogadores. Usamos a biblioteca <chronos> como recomendado. A metodologia seguiu os seguintes critérios: Na **Geração de Dados**, os jogadores foram gerados com *scores* e *timestamps* aleatórios utilizando a biblioteca <random>, garantindo que ambos os algoritmos fossem testados com a mesma entrada em cada rodada; Sobre os **Resultados**, para diminuir interferências do sistema operacional, cada teste foi repetido 5 vezes para cada tamanho de entrada. O valor registrado na tabela abaixo representa o **tempo mínimo** observado entre as repetições.

Resultados Obtidos

Tamanho (n)	Insertion Sort (ms)	Merge Sort (ms)
1.000	2	<1ms
10.000	253	5
30.000	2273	18
50.000	6302	32
75.000	14371	58
100.000	25367	78

Tabela 2: Tempos mínimos de execução para diferentes tamanhos de entrada.

Análise e Discussão

Os resultados experimentais confirmam as expectativas de complexidade assintótica dos algoritmos.

Para o *Insertion Sort*, nota-se o comportamento $O(n^2)$. Quando o tamanho da entrada (n) é pequeno, como em 1.000 elementos, o algoritmo resolve o problema quase instantaneamente (2 ms). No entanto, ao mudar o tamanho da entrada de 10.000 para 100.000 jogadores, o tempo de execução saltou de 253 ms para 25.367 ms. Esse aumento no tempo é de aproximadamente 100 vezes, o que ilustra a relação 10^2 da complexidade quadrática no pior caso e no caso médio.

Em contrapartida, o *Merge Sort* demonstrou estabilidade, evidenciando sua complexidade $O(n \log n)$. Mesmo ao ordenar 100.000 jogadores, o algoritmo concluiu a tarefa em 78 ms. O crescimento do tempo de execução do *Merge Sort* manteve-se quase linear em relação ao tamanho da entrada, com a componente logarítmica adicionando apenas uma leve curvatura ao tempo total.

Conclui-se que, embora o *Insertion Sort* seja uma abordagem viável (e de fácil implementação) para conjuntos muito pequenos de dados ou listas quase ordenadas, ele é inviável para o cenário completo proposto. O uso de um algoritmo de divisão e conquista como o *Merge Sort* mostrou-se necessário para garantir que o sistema de *matchmaking* seja eficiente para um grande número de jogadores.